

Dokumentasjon for `tia_project.py`

Oversikt

`tia_project.py` inneholder `TIAProject`-klassen som administrerer prosjekter i Siemens TIA Portal. Klassen tilbyr funksjoner for å starte TIA Portal, opprette eller åpne prosjekter, legge til enheter fra JSON-data, og konfigurere disse enhetene i prosjektet.

```
import clr # pip install pythonnet
__ref = "C:\\Program Files\\Siemens\\Automation\\Portal V19\\PublicAPI\\V19\\Siemens.Engineering.dll"
clr.AddReference(__ref)
```

Viktig og legge til PublicAPI i hver av de som trenger i da de fleste filer.

Avhengigheter

- `clr`: For å bruke .NET-biblioteker i Python (via `pythonnet`).
- `Siemens.Engineering`: Siemens TIA Portal API for å samhandle med TIA Portal.
- `json`: For å håndtere JSON-data.
- `Tia_portal_Openness.models.device_manager`: For å administrere enheter i TIA Portal-prosjektet.
- `Tia_portal_Openness.models.network_manager`: For å administrere nettverksrelaterte funksjoner.
- `Tia_portal_Openness.models.compile_manager`: For å håndtere kompilering av prosjektet.

Klasse: `TIAOpennessBase`

Metoder

`__init__(self, project_path, project_name)`

Initialiserer baseklassen med prosjektets filsti og navn. Den setter også opp interne variabler som brukes til å holde referanser til TIA Portal-instansen og prosjektet.

Klasse: `TIAProject`

`TIAProject`-klassen arver fra `TIAOpennessBase` og gir funksjonalitet for å samhandle med TIA Portal-prosjekter.

Metoder

TIA Portal Management

Start TIA Portal

Project Path:

C:\lgr\TIA

Project Name:

PythonTest11

Start TIA

`__init__(self, project_path, project_name)`

Konstruktøren initialiserer `TIAProject`-klassen og oppretter instanser av `DeviceManager`, `NetworkManager`, og `CompilerManager`.

`start_tia(self)`

Starter TIA Portal med brukergrensesnittet aktivert.

`create_project(self)`

Oppretter et nytt prosjekt i TIA Portal basert på filstien og prosjektets navn. Hvis prosjektet allerede eksisterer, åpnes det eksisterende prosjektet.

`open_existing_project(self)`

Åpner et eksisterende prosjekt basert på prosjektets filsti og navn. Hvis prosjektet ikke kan åpnes, vises en feilmelding.

`add_devices_units_from_json(self, json_data)`

Legger til enheter i prosjektet basert på en JSON-fil som inneholder enhetsdata. Denne metoden oppretter enhetene og legger til nødvendige I/O-kort.

`add_and_configure_devices(self, devices, ip_addresses)`

Legger til og konfigurerer enheter i prosjektet basert på enhetsdata og IP-adresser. Denne metoden administrerer både enheter og nettverksgrensesnitt.

Typiske Error

- Det tar tid å starte tia portal og åpne så kan hende må vente.
 - Legge til device som allerede finnes vil stoppe hele json og `break`
 - Bruker feil i excel kan oppstå
 - Kan hende er en feil i mapping så anbefaler å legge direkte til i `.json` filen
-

Dokumentasjon for `device_manager.py`

Oversikt

`device_manager.py` inneholder `DeviceManager`-klassen som administrerer enheter i et TIA Portal-prosjekt ved hjelp av Siemens TIA Portal Openness API. Denne klassen håndterer sammenfletting av enhetsmappings, oppretting av nye enheter, konfigurering av nettverksgrensesnitt og andre enhetsrelaterte operasjoner.

Avhengigheter

- `clr`: For å bruke .NET-biblioteker i Python (via `pythonnet`).
- `json`: For å laste og lagre enhetsmappings i JSON-format.
- `os`: For operativsystemrelaterte funksjoner som filstier.
- `Siemens.Engineering`: Siemens TIA Portal API for å samhandle med TIA Portal.
- `Tia_portal_Openness.models.network_manager`: For å administrere nettverksrelaterte funksjoner.

Klasse: `DeviceManager`

Metoder

`__init__(self)`

Konstruktøren initialiserer `DeviceManager`-klassen, setter opp filstier til JSON-mappings, og laster inn og fletter disse mappingene.

merge_and_load_mappings(self)

```
class DeviceManager:
    def __init__(self):
        self.file_paths = {
            "iocards": "C:\\Users\\gr\\PycharmProjects\\pythonProject\\Tia_portal_Openness\\models\\mapping_iocards.json",
            "cpucards": "C:\\Users\\gr\\PycharmProjects\\pythonProject\\Tia_portal_Openness\\models\\mapping_cpucards.json",
            "riocards": "C:\\Users\\gr\\PycharmProjects\\pythonProject\\Tia_portal_Openness\\models\\mapping_riocards.json"
        }
        self.merged_file_path = "C:\\Users\\gr\\PycharmProjects\\pythonProject\\Tia_portal_Openness\\models\\merged_mapping.json"
```

legg til egne json mappings.

Laster inn individuelle enhetsmappings fra JSON-filer, fletter dem sammen og returnerer det som en enkelt dictionary. Denne fletteprosessen lagres også som en ny JSON-fil.

add_to_map(self, name, order_number, type_card)

Legger til en ny enhet i den relevante mappingen basert på `type_card`. Oppdaterer både den originale JSON-filen og den flettede mappingen.

save_mapping_to_file(self, mapping)

Lagrer den oppdaterte enhetsmappingsdictionaryen til den flettede JSON-filen.

get_mapping(self, mapping_type=None)

Returnerer en spesifikk mapping hvis `mapping_type` er gitt, ellers returnerer den den flettede mappingen.

find_device_by_name(self, myproject, device_name)

Søker etter en enhet i prosjektet basert på navnet. Returnerer enhetsobjektet hvis det finnes, ellers kaster det en `ValueError`.

Viktig å forstå at Device Name aldri er et IO-kort men RIO, PLC, osv...

```
create_device(self, myproject, order_number, device_name, item_name,  
ip_address, subnet_masks)
```

Oppretter en enhet i prosjektet basert på det oppgitte ordrenummeret. Konfigurerer også nettverksgrensesnittene til enheten.

```
configure_network_interfaces(self, device, ip_addresses,  
subnet_masks)
```

Konfigurerer nettverksgrensesnittene til en enhet ved å tildele IP-adresser og subnet-masker.

```
add_device_cards(self, myproject, device_name, order_number, name,  
slot, potential)
```

Legger til I/O-kort til en enhet i et gitt spor og konfigurerer potensialgrupper (Light/Dark) basert på `potential`.

```
list_all_devices(self, myproject)
```

Lister opp alle enheter i prosjektet og deres tilknyttede enhetselementer (device items).

```
set_potential_group(self, myproject, device_name, device_item_name,  
potential_group_value)
```

Setter potensialgruppen for en enhetselement (device item) til enten `Light` eller `Dark`.

```
delete_device(self, myproject, device_name)
```

Sletter en enhet fra prosjektet basert på enhetens navn.

Dokumentasjon for `main.py`

Bruk av web-app

- `uvicorn Tia_portal_Openness.main:app --reload`
- `npm start` i prosjekt lokasjon

Oversikt

Med forståelse av `main.py` kan du åpne å skjønne bruken av de forskjellige funksjonene i programmet

`main.py` er en FastAPI-applikasjon som håndterer filopplasting, prosessering av Excel-ark, og oppretting og konfigurering av enheter i et Siemens TIA Portal-prosjekt. Applikasjonen inkluderer API-endepunkter som tillater brukere å laste opp Excel-filer, prosessere data fra disse filene, og generere tilhørende enheter basert på innholdet i filene.

Upload Excel File

Velg fil 20180101_K...lftunnelen.xlsx Upload

Enter Data for Each Sheet

UCA101

RIO(Ordernumber (forhold til mapping))

0.0.0.0

255.0.0.0

UCA102

Device Type

IP Address

Subnet Mask

Avhengigheter

- `fastapi`: Web-rammeverk for å bygge API-endepunkter.
- `pandas`: For å håndtere Excel-filer og databehandling.
- `json`: For å lagre og laste inn JSON-data.
- `pydantic`: For å validere inndata til API-endepunktene.
- `typing`: For å spesifisere typer i funksjoner og datamodeller.
- `Tia_portal_Openness.models.tia_project`: Importerer `TIAProject`-klassen for å håndtere TIA Portal-prosjektet.

API-endepunkter

`/upload-file/` (POST)

Dette endepunktet håndterer opplasting av en Excel-fil. Filen blir lagret midlertidig på serveren, og de tilgjengelige arkene (sheets) i filen blir returnert som en liste.

Parametre

- `file`: Filen som lastes opp, forventes å være av typen `UploadFile`.

Returverdi

- En liste over arknavn i Excel-filen.

Feilhåndtering

- Returnerer HTTP 400-feil hvis filen ikke kan lastes opp eller behandles.

/process-sheets/ (POST)

Dette endepunktet prosesserer data fra de spesifiserte arkene i en tidligere opplastet Excel-fil. Dataene blir brukt til å generere JSON-strukturer som beskriver enheter og deres konfigurasjoner. Disse JSON-dataene lagres til fil og blir deretter brukt til å opprette og konfigurere enheter i TIA Portal-prosjektet.

Inndata

- **SheetInput**: En Pydantic BaseModel som inneholder en liste over arknavn (**sheets**) og tilhørende inndata (**inputData**) for hver ark.

Returverdi

- En melding som indikerer at dataene ble prosessert og lagret vellykket, sammen med de prosesserte JSON-dataene.

Feilhåndtering

- Returnerer HTTP 404-feil hvis den opplastede filen ikke finnes.
- Returnerer HTTP 500-feil for generelle feil under prosessering av arkene.

process_json_data(data)

En funksjon som tar inn JSON-data generert fra Excel-arkene, og bruker disse dataene til å opprette enheter i TIA Portal-prosjektet.

```
main.py  all_sheets_devices.json  test2.py
1  {
2      "UCA101": {
3          "Device Name": "UCA101",
4          "IP-addr": "2.2.2.2",
5          "SubnetMask": "255.255.0.0",
6          "DeviceType": "RIOST",
7          "Cards": {
8              "1": {
9                  "Type": "DI",
10                 "BaseUnit": "WH",
11                 "Adresses": {
12                     "0": {
13                         "Tag": "SOS01SKAP",
14                         "Signal": "XFS",
15                         "Objtype": "52. Skap status"
16                     },
17                     "1": {
18                         "Tag": "SOS01SKAP",
19                         "Signal": "XNETIF",
20                         "Objtype": "52. Skap status"
21                     },
22                     "2": {
23                         "Tag": "SOS01SKAP",
24                         "Signal": "XOVERSP",
25                         "Objtype": "52. Skap status"
26                     },
27                     "3": {
28                         "Tag": NaN,
29                         "Signal": NaN,
30                         "Objtype": NaN
31                     }
32                 }
33             }
34         }
35     }
```

Parametre

- **data**: En dictionary som inneholder enhetsdata hentet fra Excel-arkene.

Feilhåndtering

- Funksjonen skriver ut feilmeldinger hvis en enhet eller et kort ikke kan opprettes eller tildeles riktig.

Intern Logikk og Flyt

1. **Filoplasting:**
 - Filen lastes opp via `/upload-file/`-endepunktet, lagres midlertidig, og arknavnene i filen returneres.
 2. **Prosessering av Ark:**
 - Når arknavn og tilhørende inndata er spesifisert i `/process-sheets/`-endepunktet, leses hvert ark, og dataene i arket behandles for å identifisere enheter og deres tilhørende I/O-kort.
 - Enhetene og kortene blir strukturert i en JSON-formatert dictionary som lagres til fil.
 3. **Oppretting og Konfigurering av Enheter:**
 - JSON-dataene brukes til å opprette enheter i TIA Portal-prosjektet. Hver enhet får tildelt I/O-kort og nettverksinnstillinger basert på dataene i Excel-arkene.
 - Funksjonen `process_json_data` håndterer denne prosessen, og det logges for å sikre sporbarhet av prosessen.
-

Dokumentasjon for Tunnel Panel Generator

Oversikt

Dette Python-skriptet genererer XML-konfigurasjoner for tunnelpaneler basert på tunnelens lengde og ulike systemdata som hentes fra CSV-filer. Koden gir mulighet for å lage flere typer paneler, inkludert hovedløp og teknisk layout for tunneler. Skriptet håndterer også formatering og korrigering av CSV-data, og genererer XML-filer som kan brukes i tilknyttede applikasjoner. For tilpasning av flere elementer er det bare å legge in flere if setninger, og lokasjon til disse.

Avhengigheter

Koden bruker følgende Python-moduler:

- `xml.etree.ElementTree`: Brukes til å bygge XML-strukturen.
- `xml.dom.minidom`: Brukes til å formattere XML-strengen for lesbarhet.
- `pandas`: Brukes til å lese og manipulere CSV-filer.

Klasser

Panel

`Panel`-klassen representerer et panel i XML-formatet. Denne klassen er ansvarlig for å sette opp panel-egenskapene og legge til figurer og referanser.

Metoder

- `__init__(self, width, height)`: Initialiserer et nytt `Panel`-objekt med gitt bredde og høyde, og oppretter panel- og form-elementer.
- `_add_properties(self, width, height)`: Privat metode som legger til egenskapene til panelet, inkludert størrelse, bakgrunnsfarge, referansepunkt, etc.
- `add_shape(self, shape)`: Legger til en figur (shape) til panelet.
- `add_reference(self, reference)`: Legger til en referanse til panelet.
- `to_pretty_xml(self)`: Returnerer XML-strukturen som en formattet streng.

Shape

`Shape`-klassen representerer en figur i panelet, som f.eks. et rektangel eller en linje.

Metoder

- `__init__(self, shape_type, name, serial_id, properties)`: Initialiserer en figur med gitt type, navn, `serial_id` og egenskaper.

Reference

`Reference`-klassen representerer en referanse til et annet panel eller objekt i panelet.

Metoder

- `__init__(self, name, reference_id, location, tab_order, filename, dollar_parameters=None)`: Initialiserer en referanse med nødvendig informasjon.
- `_add_properties(self, location, tab_order, filename, dollar_parameters)`: Privat metode som legger til egenskaper for referansen.
- `to_element(self)`: Returnerer referansen som et XML-element.

Funksjoner

create_panel_xml

Genererer XML for et panel basert på tunnelens lengde og paneldimensjoner. Oppretter figurer som rektangler og meter-markører.

Parametre

- `tunnel_length_meters`: Lengden på tunnelen i meter.
- `panel_width`: Bredden på panelet.
- `panel_height`: Høyden på panelet.
- `start_at_zero`: Boolsk verdi som bestemmer om panelet skal starte fra null eller ikke.

Returverdi

- Returnerer en formattert XML-streng for panelet.

save_xml_to_file

Lagrer en XML-streng til en fil.

Parametre

- `xml_str`: XML-strengen som skal lagres.
- `filename`: Filnavnet som XML-en skal lagres som.

correct_and_read_csv

Korrigerer kolonneantallet i en CSV-fil og leser dataene som en DataFrame.

Parametre

- `file_path`: Stien til den originale CSV-filen.
- `output_path`: Stien til den korrigerede CSV-filen.
- `num_columns`: Antall kolonner som CSV-filen skal ha.
- `delimiter`: Delimiter som brukes i den originale CSV-filen.
- `target_delimiter`: Delimiter som skal brukes i den korrigerede CSV-filen.

Returverdi

- Returnerer en Pandas DataFrame med korrigerede data.

extract_columns_as_dict

Ekstraherer kolonner fra en CSV-fil og returnerer dem som en dictionary.

Parametre

- `file_path`: Stien til CSV-filen.
- `output_path`: Stien til den korrigerte CSV-filen.

Returverdi

- Returnerer en dictionary der nøklene er verdiene fra `DP`-kolonnen og verdiene er fra `DPT`-kolonnen.

`create_reference_hovedlop1_xml`

Genererer XML for et hovedløp basert på data fra en dictionary og en bakgrunnsfil.

I denne kan du selv legge til `elif` setninger med "value" fra CSV'en, også legge til filename for png. Her må du også legge til `location` basert på x og y aksen.

Parametre

- `data_dict`: Dictionary som inneholder `DP` og `DPT`-data.
- `background_filename`: Filnavnet for bakgrunnsbildet.

Returverdi

- Returnerer en formattert XML-streng for hovedløpet.

`create_hovedlop_teknisk_xml`

Genererer XML for teknisk hovedløp basert på data fra en dictionary og en bakgrunnsfil.

I denne kan du selv legge til `elif` setninger med "value" fra CSV'en, også legge til filename for png. Her må du også legge til `location` basert på x og y aksen.

Parametre

- `data_dict`: Dictionary som inneholder `DP` og `DPT`-data.
- `background_filename`: Filnavnet for bakgrunnsbildet.

Returverdi

- Returnerer en formattert XML-streng for det tekniske hovedløpet.

`main`

Hovedfunksjonen som styrer hele prosessen med å generere panelene og lagre dem som XML-filer. Den inkluderer brukerinput for spesifikke valg.

```
# Her tar du inn CSV'en
```

```
input_file_path =  
'C:\\Users\\gr\\PycharmProjects\\pythonProject\\XML_tunnel\\F15S01  
_HMI_TAGS_Valderøytunnelen.csv'  
  
output_file_path =  
'C:\\Users\\gr\\PycharmProjects\\pythonProject\\XML_tunnel\\F15S01  
_HMI_TAGS_Valderøytunnelen1.csv'
```

Når du bruker input på csv er det viktig at det er 2 separate filer. Her ser du forskjellen på at den ene filen slutter på en "1".